

Lab 4: Mastering Code Understanding and Documentation Prompts

Objective

Learn how to use specialized prompts with GitHub Copilot to quickly understand, document, and work with existing codebases. You'll practice using targeted prompts that help you analyze code, create documentation, and onboard to new projects efficiently.

What You Will Learn in This Lab

- Using specific prompt patterns to analyze and understand code
- Generating comprehensive project documentation with AI assistance
- Creating onboarding materials and usage guides
- Getting architectural insights and testing recommendations
- Building interactive learning materials like quizzes
- Mastering prompt engineering for code analysis tasks

Prerequisites: You should have a codebase to work with. You can either use your own codebase or use the sample codebase provided.

Steps

Step 1: Project Setup

1. Choose a codebase to analyze:
 - **Use your own project** (recommended if you have one)
 - **Use the provided sample codebase:** <https://github.com/DevOnGlobal/ai-playground-exercises>
2. If using the sample codebase:

```
bash

git clone https://github.com/DevOnGlobal/ai-playground-exercises
...cd ai-playground-exercises
```

3. Open the project in VS Code with Copilot enabled
4. (Optional) Create a new file called `analysis-results.md` to save your outputs

Step 2: Code Explanation and Understanding

Let's start with basic code understanding prompts.

1. Explain Code in Simple Terms

- Select a complex function or code block
- Use the prompt:
- Try variations:
 -
 -
 -

2. Identify Key Components

- Select your entire project or main files
- Use the prompt:
- Document the response in your analysis file

3. Function and Class Summary

- Use the prompt:
- Try with different scopes:
 - Individual files:
 - Specific classes:

Step 3: Project Onboarding and Documentation

Now let's create comprehensive onboarding materials.

1. Create an Onboarding Guide

- Use:
- Follow up with:
- Ask:

2. Generate Quickstart Instructions

- Use:
- Try:
- Ask:

3. User-Facing Documentation

- Use:

- For APIs:
- For libraries:

Step 4: Operational Understanding

Let's understand how to run and work with the code.

1. Execution and Functionality

- Use:
- Ask:
- Try:

2. Task Identification

- Use:
- Ask:
- Try:

Step 5: Testing and Quality Analysis

Focus on testing and code quality insights.

1. Testing Recommendations

- Use:
- Ask:
- Try:

2. Edge Cases and Scenarios

- Use:
- Ask:
- Try:

3. Code Review

- Use:
- Ask:
- Try:

Step 6: Architecture and Design Analysis

Understand the bigger picture of your codebase.

1. Architecture Overview

- Use:
- Ask:
- Try:

2. Dependency Analysis

- Use:
- Ask:
- Try:

Step 7: Interactive Learning and Assessment

Create materials to test understanding.

1. Generate Quiz Questions

- Use:
- Try variations:
 -
 -
 -

2. Interactive Exploration

- Ask:
- Try:
- Use:

Step 8: Specialized Analysis Prompts

Practice with more advanced analytical prompts.

1. Performance Analysis

- Use:
- Ask:
- Try:

2. Security Review

- Use:
- Ask:
- Try:

3. Maintainability Assessment

- Use:
- Ask:

Step 9: Documentation Generation Practice

Create comprehensive documentation using your analysis.

1. README Generation

- Use:
- Include:

2. Technical Documentation

- Use:
- Ask:

3. Troubleshooting Guide

- Use:
- Ask:

Step 10: Advanced Prompt Combinations

Practice combining multiple prompt patterns.

1. Comprehensive Analysis

- Try:

2. Role-Based Prompts

- Use:
- Try:
- Ask:

Key Takeaways to Discuss

- Different prompt patterns serve different analysis purposes
- Specific, targeted prompts yield more useful results than generic questions
- Combining multiple prompts creates comprehensive understanding
- AI can help create learning materials to reinforce understanding
- Documentation generation can be significantly accelerated with proper prompts

- Code analysis prompts help identify issues you might miss manually

Best Practices for Code Analysis Prompts

- **Be specific about scope:** Target individual functions, files, or the entire project
- **Specify your role/level:** "As a junior developer" vs "As a senior architect"
- **Ask for examples:** Request concrete examples rather than just explanations
- **Iterate and refine:** Start broad, then ask follow-up questions for specifics
- **Request different formats:** Ask for diagrams, lists, code examples, or documentation

Challenge Extensions

Try these advanced scenarios:

- Compare two different codebases and analyze their approaches
- Create migration guides between different versions of a project
- Generate training materials for specific technologies used in the project
- Create automated code review checklists based on the codebase analysis

Next Steps

Practice these prompt patterns on different types of projects (web apps, APIs, libraries, etc.) to see how Copilot adapts its responses to different technologies and architectural patterns.